

deeplearning.ai

Error Analysis

Carrying out error analysis

- If you're trying to get a learning algorithm to do a task that humans can do and if your learning algorithm is not yet at the performance of a human, then manually examining mistakes that your algorithm is making, can give you insights about what to do next.
- This process is called error analysis.

Cat classifier Example

Look at dev examples to evaluate ideas





You have achieved:

- 90% accuracy or 10% error on dev set.
- You're not happy with the result.

- Looking at misclassified images, you notice that some dogs are miscategorized as cats. (Some dogs really resemble cats)

- Should you try to make your cat classifier do better on dogs? → may take 3 months.

Error analysis: may help you decide if worth the effort

- Get ~100 mislabeled dev set examples.
- Count up how many are dogs.

- Worth the effort?

Case 1

- If out of 100 mislabeled images only 5 of them are misclassified dogs, that means 5% of the cause of errors are dog images.
- Even if you correct the labels of those 5 dog images, the 10% total error will drop to 9.5%.
- This gives you a "ceiling" about the improvement you may expect (0.5%). You may decide it is not worth the effort.

Case 2

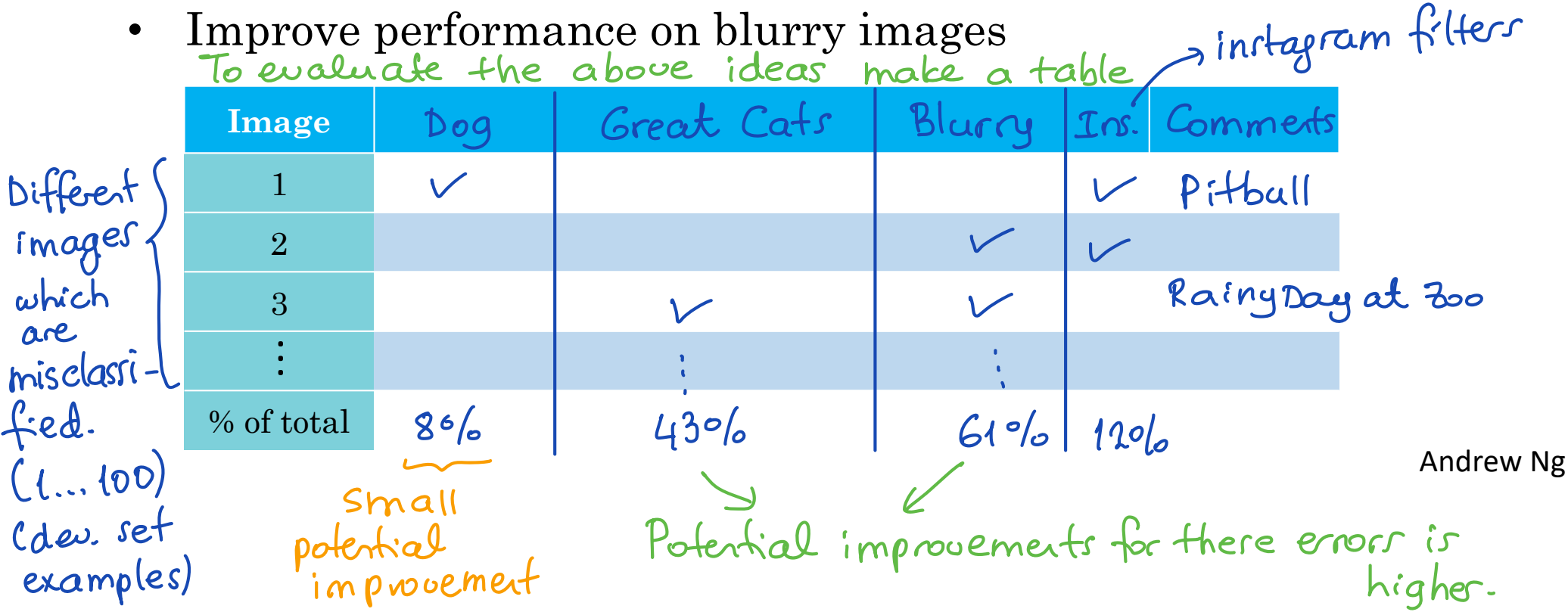
- If 50 of the mislabeled images are originating from dog images and if they're corrected, the overall error will go from 10% to 5%. ⇒ worth the effort

- Error analysis is helpful to decide which error types to work on. Which ones are worth the effort?

Evaluate multiple ideas in parallel

Ideas for cat detection:

- Fix pictures of dogs being recognized as cats
- Fix great cats (lions, panthers, etc..) being misrecognized
- Improve performance on blurry images



Andrew Ng



deeplearning.ai

Error Analysis

Cleaning up Incorrectly labeled data

Incorrectly labeled examples : cat classification

x 

y	1	0	1	1	0	1	1

incorrect label

DL algorithms are quite robust to random errors in the training set.

- DL algorithms are less robust to systematic errors.
- How about incorrectly labeled examples in the dev set or test set?

Andrew Ng

- Add an extra column to count the # examples where the label Y was incorrect.

Error analysis

Image	Dog	Great Cat	Blurry	Incorrectly labeled	Comments
...					
98				✓	Labeler missed cat in background
99		✓			
100				✓	Drawing of a cat; Not a real cat.
% of total	8%	43%	61%	6%	

Overall dev set error ..... 10% | 2%

Errors due incorrect labels .... 0.6% (6/10) | 0.6%

Errors due to other causes ---- 9.4% | 1.4%

relatively small

incorrect labels are not worth correcting

a large portion of error is due to mislabeled samples. worth correcting

Goal of dev set is to help you select between two classifiers A & B.

Andrew Ng

- If A has 2.1% error and B has 1.9% error, but you can't say B is better since there are mislabeled samples. ⇒ correct the labels

Correcting incorrect dev/test set examples

- Apply same process to your dev and test sets to make sure they continue to come from the same distribution
- Consider examining examples your algorithm got right as well as ones it got wrong. Some of the correctly classified examples may also need to be corrected.
- Train and dev/test data may now come from slightly different distributions.
  - You may decide to correct the mislabeled samples in your dev/test set instead of the much larger training set.
  - It is very important that dev and test sets come from the same distribution.

Andrew Ng



deeplearning.ai

Error Analysis

Build your first system quickly, then iterate

Speech recognition example

There are a lot of things to improve:

- Noisy background
  - Café noise
  - Car noise
- Accented speech
- Far from microphone
- Young children's speech
- Stuttering : uh , ah, um
- …

Guideline:
Build your first system quickly, then iterate

- Set up dev/test set and metric
- Build initial system quickly
- Use Bias/Variance analysis & Error analysis to prioritize next steps.

- How to choose which direction to focus on?

Andrew Ng

Build system quickly, then iterate

Depending on the area of application, the guideline below will help you prioritize when you build your system.

Guideline

1. Set up development/ test set and metrics
   - Set up a target
2. Build an initial system quickly
   - Train training set quickly: Fit the parameters
   - Development set: Tune the parameters
   - Test set: Assess the performance
3. Use Bias/Variance analysis & Error analysis to prioritize next steps

e.g. if error analysis reveals that most of the errors are caused by being far from microphone in speech recognition, you can focus on that problem.



deeplearning.ai

Mismatched training and dev/test data

Training and testing on different distributions

- Deep learning algorithms work best when we use enough labeled data in the training set.
- Sometimes, training set may contain whatever data is available.
- Most of the training data may not come from the same distribution as dev and test sets.
- There are some best practices for dealing with when training and test distributions differ from each other.

Cat app example

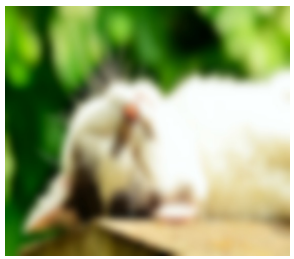
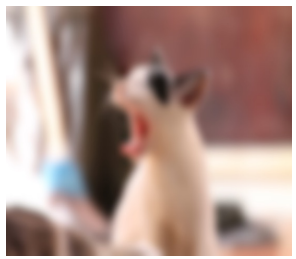
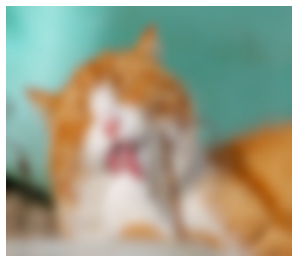
Data from webpages





sharp, high-res images
collected ≈ 200.000 samples

Data from mobile app

blury images
collected ≈10.000 samples

- Goal: The final cat recognition system will do well on images uploaded from a mobile app.
- Dilemma: We have a small dataset (10.000) from mobile app, but a larger dataset (200.000) from another distribution.
- You don't want to use just 10000 images for training since it will be a small dataset.

Andrew Ng

- Using the 200.000 images might be useful, but they don't come from the distribution you want.
- Solution 1: Put both datasets together, which will give 210.000 images. Then, randomly shuffle into a train, dev and test set.



advantage: train, dev and test sets come from the same distribution.

disadvantage: most of the dev set examples will come from the web page distribution of images, rather than the mobile app distribution of images.

dev set: 2500 → 2381 web page images
→ 119 mobile app images
on the average

✗ Solution 1 is not recommended.

Solution 2: Put 205.000 images in the training set, where 200.000 images are from the web, and 5000 are from the mobile app.



advantage: dev set contains images from the mobile app.

disadvantage: training distribution is different from dev and test set distributions.

Goal: To build a new product: speech activated rearview mirror for a car. (e.g. where's the nearest gas station? etc.)

Speech recognition example



- How to split the training, dev/test sets?

Training

→ Purchased data
(x, y): (audio clip, transcripts)

→ Smart speaker control

→ Voice keyboard

... collected 500000 utterances from these resources

Dev/test

Speech activated rearview mirror

20,000 samples
10,000 dev / 10,000 test set

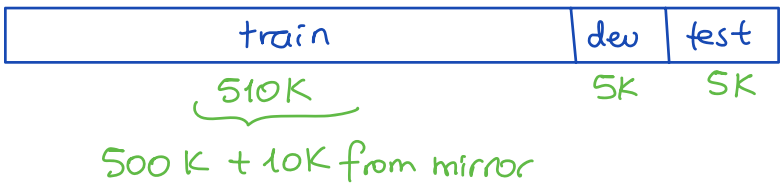
- Training and dev/test distributions will be different.



Andrew Ng

- But performance on dev/test is what we really care about.

Alternatively the split can be ar follows:



- bigger training set

Training and testing on different distributions

Example: Cat vs Non-cat

In this example, we want to create a mobile application that will classify and recognize pictures of cats taken and uploaded by users.

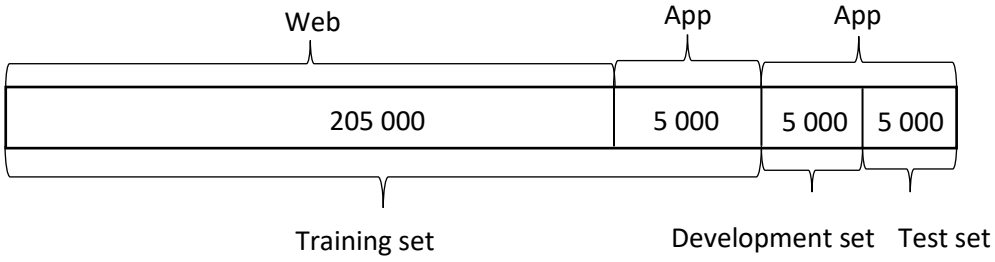
There are two sources of data used to develop the mobile app. The first data distribution is small, 10 000 pictures uploaded from the mobile application. Since they are from amateur users, the pictures are not professionally shot, not well framed and blurrier. The second source is from the web, you downloaded 200 000 pictures where cat's pictures are professionally framed and in high resolution.

The problem is that you have a different distribution:

1- small data set from pictures uploaded by users. This distribution is important for the mobile app.
2- bigger data set from the web.

The guideline used is that you have to choose a development set and test set to reflect data you expect to get in the future and consider important to do well.

The data is split as follow:



The advantage of this way of splitting up is that the target is well defined.

The disadvantage is that the training distribution is different from the development and test set distributions. However, this way of splitting the data has a better performance in long term.



deeplearning.ai

Mismatched training and dev/test data

Bias and Variance with mismatched data distributions

- Estimating the bias and variance of your learning algorithm really helps you prioritize what to work on next.
- But the way you analyze bias and variance changes when your training set comes from a different distribution than your dev and test sets.

Cat classifier example

Assume humans get ≈ 0% error.

Training error ... 1%
Dev error 10%

} ↓9% ⇒ high variance problem

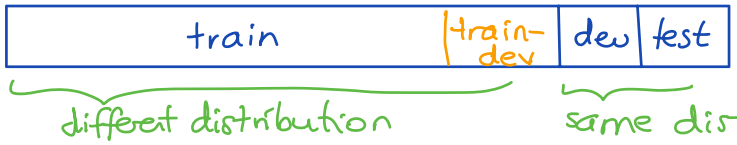
- If your dev data comes from the same distribution as your training set
- But if dev set has a different distribution from training set, you can't say there's high variance. (maybe training set was easy with high-res images, and dev set had blurry images)

Training-dev set: Same distribution as training set, but not used for training

Andrew Ng

- Problem with this analysis: when we went from the training error to the dev error, two things changed at a time:
  1) The algorithm saw the data in the training set but not in the dev. set
  2) The distribution of data in the dev set is different.

- To tease out the above effects, it will be useful to define a new piece of data called training-dev set, with the same distribution as the training set.
- It is a piece of the training set but you don't explicitly train a NN on this set.
- Previously we had set up our data as follows:



- Train the NN on the "train" dataset.
- Carry-out error analysis on the training, training-dev and dev sets:

Train error	1%	1%
Train-dev error	9%	1.5%
Dev error	10%	10%

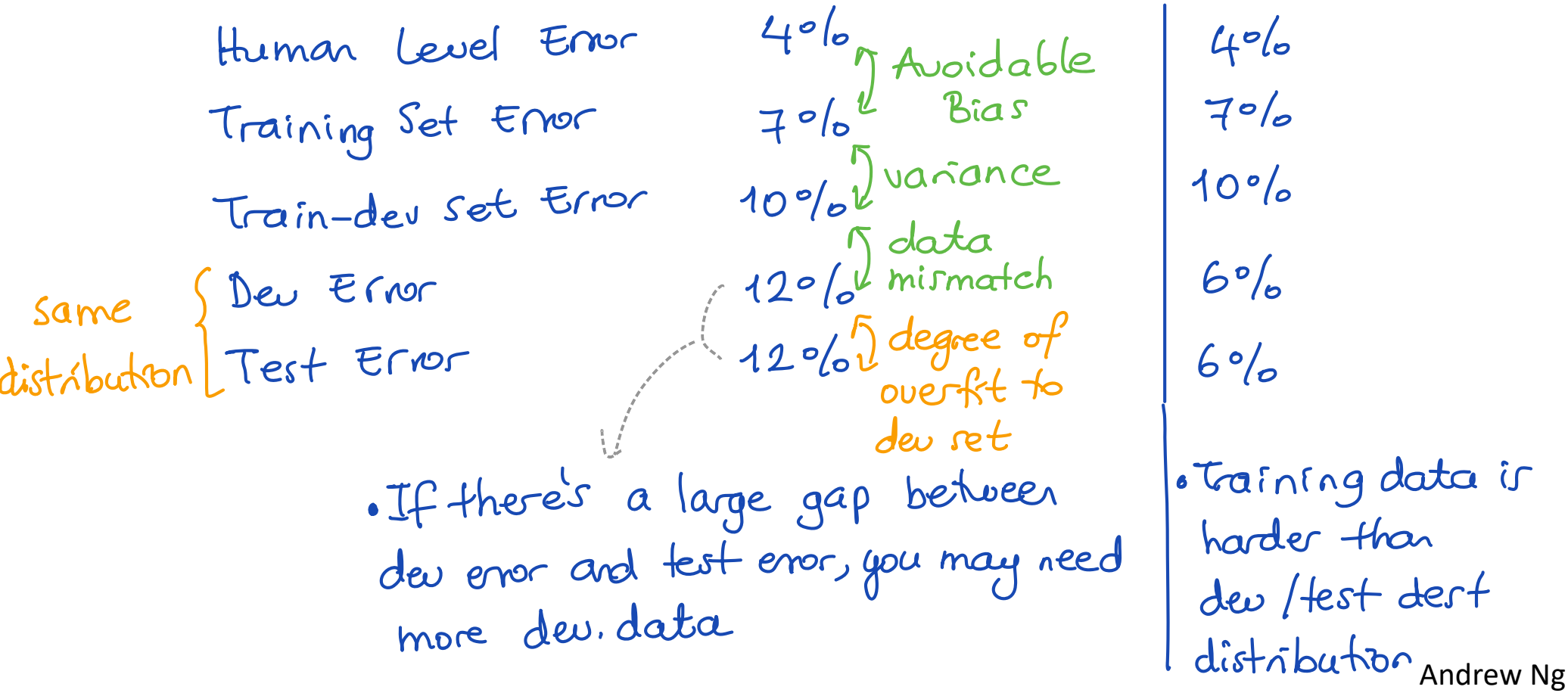
(Train error, Train-dev error: have same distribution; 1% → 9%: high variance problem; Train-dev error → Dev error: data mismatch problem; Dev error: variance)

Human Error	0%	0%
Training Error	10%	10%
Training-Dev Error	11%	11%
Dev Error	12%	20%

(0% → 10%: Avoidable Bias; 0% → 10%: Avoidable Bias problem; 11% → 20%: Data mismatch; 11%: small variance; 10%: High Bias; 20%: Bias + data mismatch)

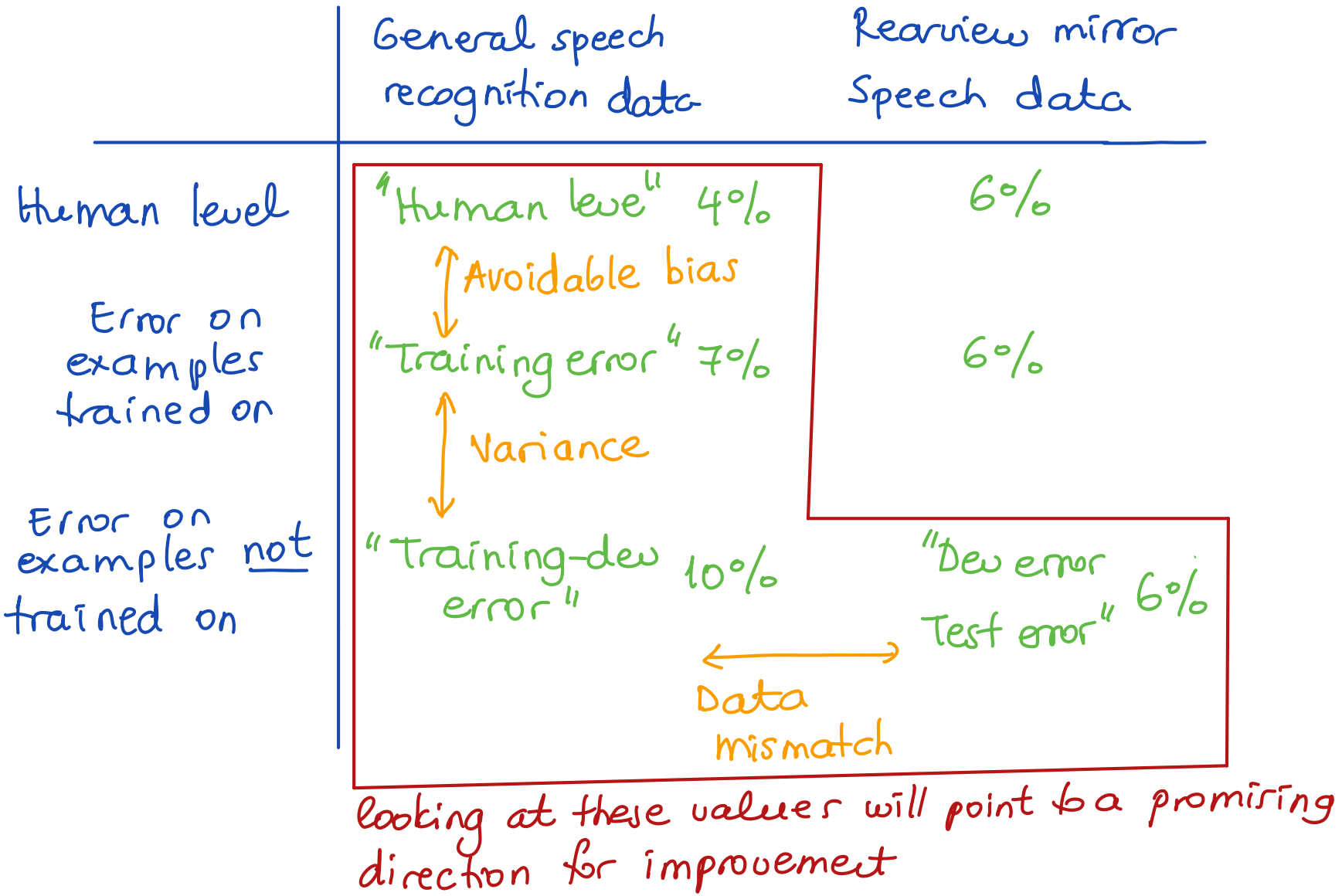
- Write out the general principles

Bias/variance on mismatched training and dev/test sets



Example: Speech activated rearview mirror

More general formulation



Andrew Ng

Bias and variance with mismatched data distributions

Example: Cat classifier with mismatch data distribution

When the training set is from a different distribution than the development and test sets, the method to analyze bias and variance changes.

	Classification error (%)					
	Scenario A	Scenario B	Scenario C	Scenario D	Scenario E	Scenario F
Human (proxy for Bayes error)	0	0	0	0	0	4
Training error	1	1	1	10	10	7
Training-development error	-	9	1.5	11	11	10
Development error	10	10	10	12	20	6
Test error	-	-	-	-	-	6

Scenario A

If the development data comes from the same distribution as the training set, then there is a large variance problem and the algorithm is not generalizing well from the training set.

However, since the training data and the development data come from a different distribution, this conclusion cannot be drawn. There isn't necessarily a variance problem. The problem might be that the development set contains images that are more difficult to classify accurately.

When the training set, development and test sets distributions are different, two things change at the same time. First of all, the algorithm trained in the training set but not in the development set. Second of all, the distribution of data in the development set is different.

It's difficult to know which of these two changes what produces this 9% increase in error between the training set and the development set. To resolve this issue, we define a new subset called training-development set. This new subset has the same distribution as the training set, but it is not used for training the neural network.

Scenario B

The error between the training set and the training- development set is 8%. In this case, since the training set and training-development set come from the same distribution, the only difference between them is the neural network sorted the data in the training and not in the training development. The neural network is not generalizing well to data from the same distribution that it hadn't seen before

Therefore, we have really a variance problem.

Scenario C

In this case, we have a mismatch data problem since the 2 data sets come from different distribution.

Scenario D

In this case, the avoidable bias is high since the difference between Bayes error and training error is 10 %.

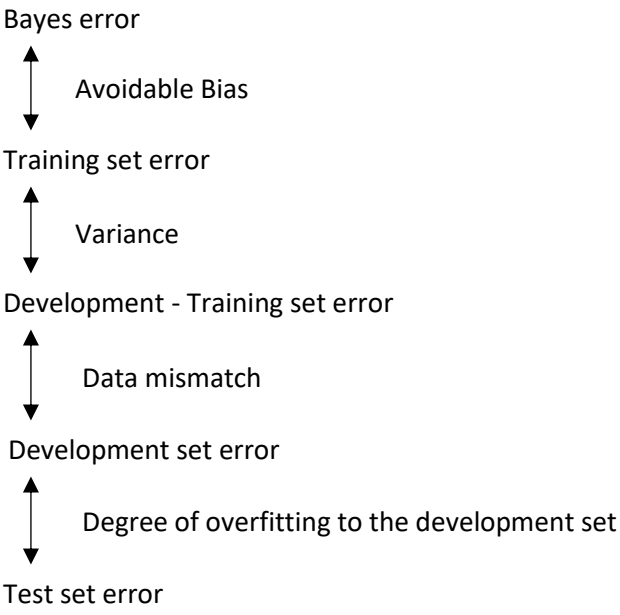
Scenario E

In this case, there are 2 problems. The first one is that the avoidable bias is high since the difference between Bayes error and training error is 10 % and the second one is a data mismatched problem.

Scenario F

Development should never be done on the test set. However, the difference between the development set and the test set gives the degree of overfitting to the development set.

General formulation





deeplearning.ai

Mismatched training and dev/test data

Addressing data mismatch

- How can we handle the data mismatch problem?

Addressing data mismatch

- Carry out manual error analysis to try to understand difference between training and dev/test sets

  e.g. dev set may have car noise

- Make training data more similar; or collect more data similar to dev/test sets

  eg. simulate noisy in-car data

Artificial data synthesis

 +  = 

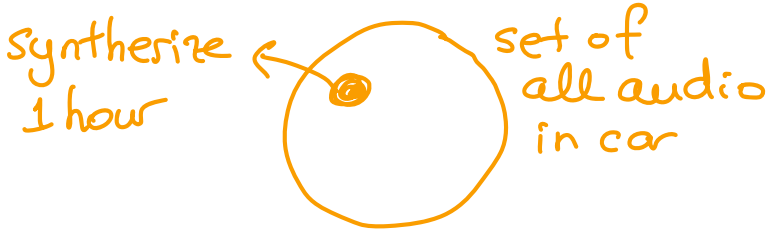
"The quick brown fox jumps over the lazy dog."

(contains all letters a-z)

10,000 hours

Car noise

1 hour of car noise



Synthesized in-car audio

- If you repeat the car noise 10000 times and add to the clean data, it will sound perfectly fine to human ear, but the network may overfit to 1 hour of car noise.

Andrew Ng

Artificial data synthesis

Car recognition:

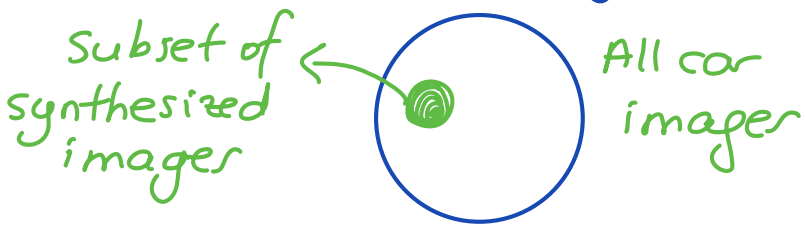






You want to recognize cars like this and put a bounding box around it.

- You can use computer graphics to simulate car images.



You may overfit to this subset → be careful

Andrew Ng

Addressing data mismatch

This is a general guideline to address data mismatch:

- Perform manual error analysis to understand the error differences between training, development/test sets. Development should never be done on test set to avoid overfitting.

- Make training data or collect data similar to development and test sets. To make the training data more similar to your development set, you can use is artificial data synthesis. However, it is possible that if you might be accidentally simulating data only from a tiny subset of the space of all possible examples.



deeplearning.ai

Learning from multiple tasks

Transfer learning

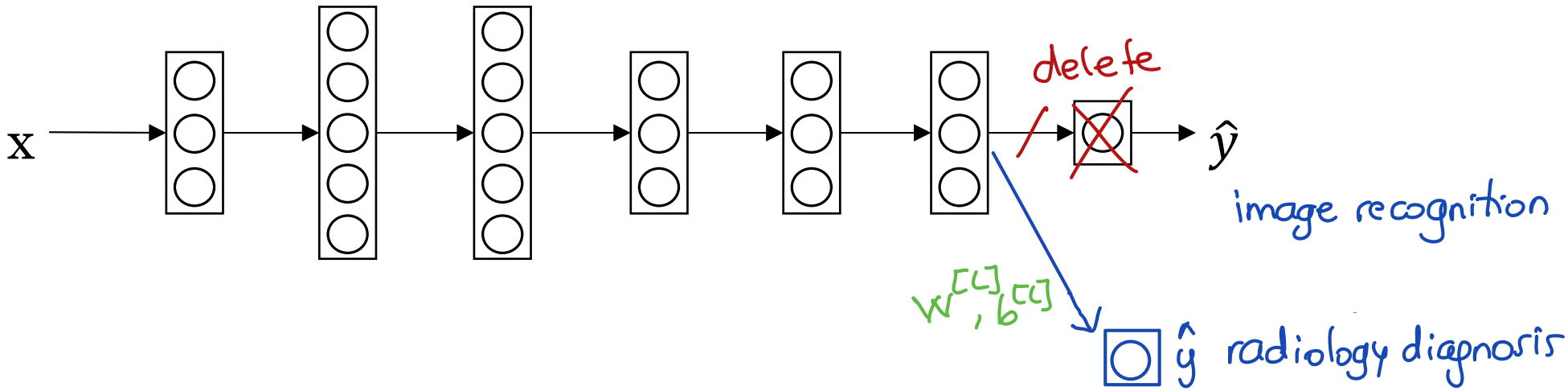
- Powerful idea in deep learning: You can take knowledge the network has learned from one task and apply that knowledge to another task.
- Example: You could have a trained network which recognizes cats and use that knowledge to help you do a better job reading x-ray scans.
- This is called transfer learning.

- First, we take a NN and train it on (x, y) pairs, where x is an image and y is some object. (e.g. cat, dog, bird etc.)
- To adapt (or transfer) what is learned to a different task, (e.g. radiology diagnosis from x-ray scans), you can:

1) delete the last output layer of the network and also delete the weights feeding into the last layer.
2) Create a new set of randomly initialized weights just for the last layer, with the new # of outputs for radiology problem.
3) Retrain the NN on the new radiology image dataset:
   - → You may retrain just the weights of the last layer if the new dataset is small. (and keep the rest of the parameters fixed)
   - → You may retrain all the layers if the dataset is large.

- If you retrain all the parameters in the neural network, the initial phase of training on image recognition is called pre-training.
- Then, if you are updating all the weights afterwards, then training on radiology data is called fine-tuning.

Transfer learning



- We took the knowledge learned from image recognition and transfered it to radiology diagnosis.
- Why does this help? : A lot of the low level features such as detecting edges, detecting curves have been learned from a very

Andrew Ng

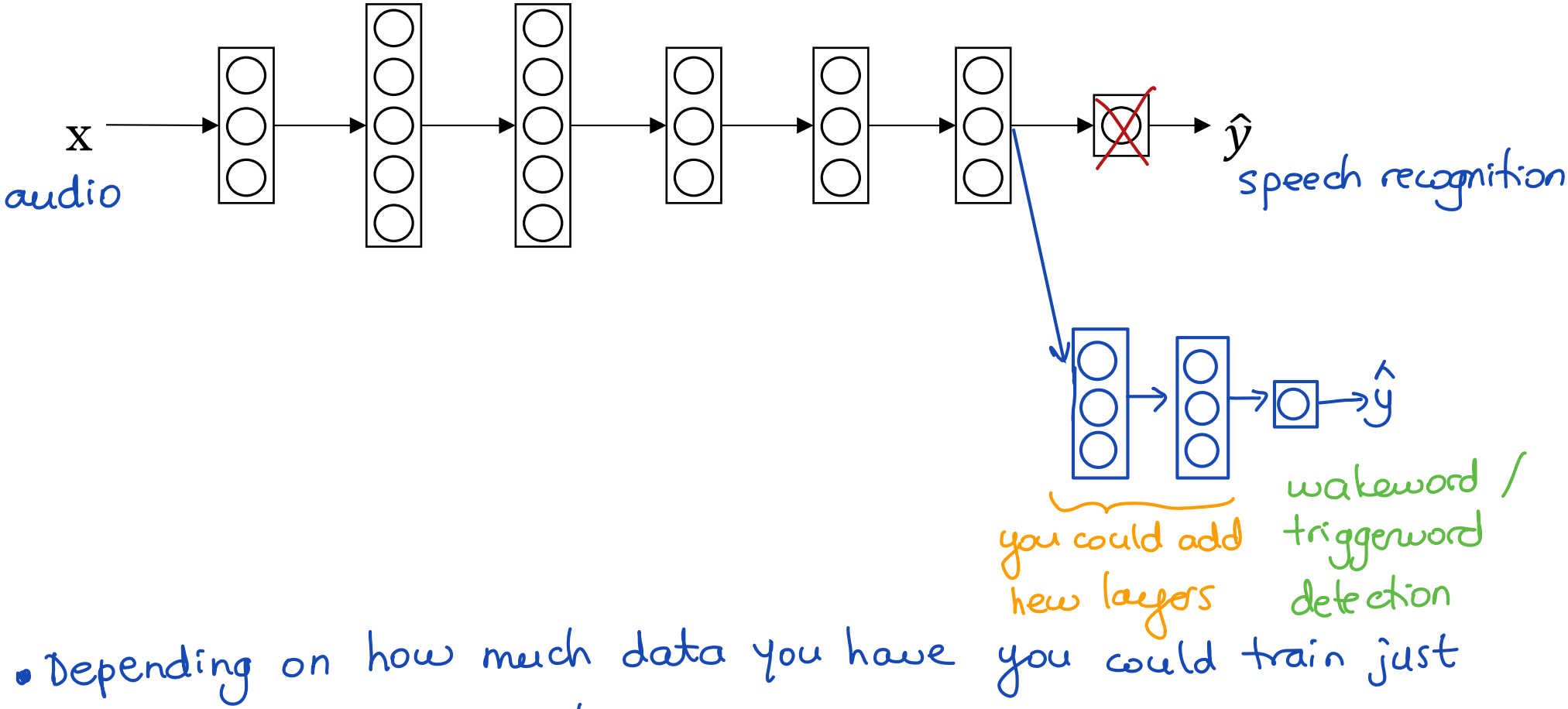
large image recognition dataset which also helps on the radiology image dataset to learn faster or with less data.

- Transfer learning makes sense when you have a lot of data for the problem you're transferring from and usually relatively less data for the problem you're transferring to.
- e.g. image recognition training data : 1.000.000 images
  radiology diagnosis data : 100 images

- e.g. speech recognition data : 10000 hours
  wakeword/triggerword data : 1 hour

- Transfer learning doesn't make sense if the opposite is true:
  - e.g. image recognition training data : 100 images
    radiology diagnosis data : 1000 images
  - Transfer learning might not hurt, but it won't give a meaningful gain either.

Transfer learning: speech recognition



- Depending on how much data you have you could train just the new layers or more layers.

Andrew Ng

When transfer learning makes sense

Transfer from task A to task B.

- Task A and B have the same input x.
- You have a lot more data for Task A than Task B.
- Low level features from A could be helpful for learning B.

Andrew Ng

Transfer Learning

Transfer learning refers to using the neural network knowledge for another application.

When to use transfer learning

- Task A and B have the same input x
- A lot more data for Task A than Task B
- Low level features from Task A could be helpful for Task B

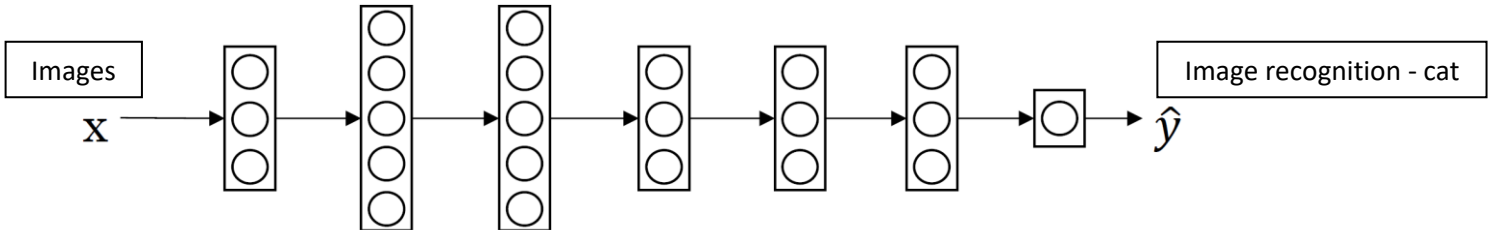
Example 1: Cat recognition - radiology diagnosis

The following neural network is trained for cat recognition, but we want to adapt it for radiology diagnosis. The neural network will learn about the structure and the nature of images. This initial phase of training on image recognition is called pre-training, since it will pre-initialize the weights of the neural network. Updating all the weights afterwards is called fine-tuning.

For cat recognition

Input x: image

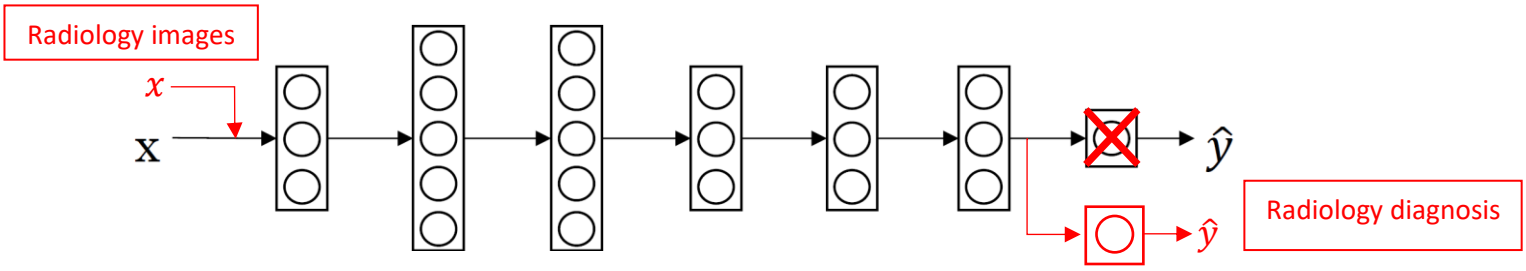
Output y – 1: cat, 0: no cat



Radiology diagnosis

Input x: Radiology images – CT Scan, X-rays

Output y :Radiology diagnosis – 1: tumor malign, 0: tumor benign



Guideline

- Delete last layer of neural network
- Delete weights feeding into the last output layer of the neural network
- Create a new set of randomly initialized weights for the last layer only
- New data set (x, y)



deeplearning.ai

Learning from multiple tasks

Multi-task learning

- In transfer learning, you have a sequential process where you learn from task A and then transfer that to task B.
- In multi-task learning, you start off simultaneously, trying to have one neural network do several things at the same time.. Each of these tasks helps with all other tasks.

Goal: Build a self-driving car.

Simplified autonomous driving example



$x^{(i)}$

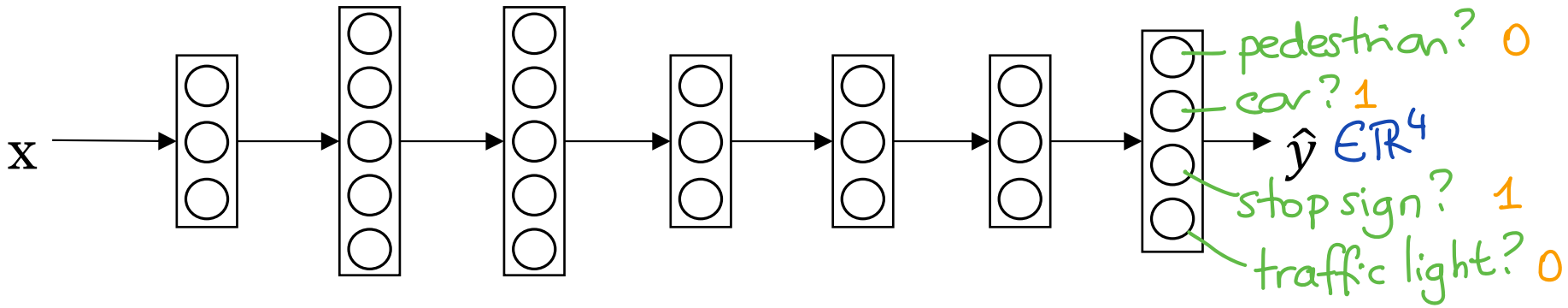
The self-driving car will detect:

	$y^{(i)}$	
Pedestrians	0 → no pedestrians	(4,1)
Cars	1 → there's a car	
Stop signs	1 → there's a stop sign	
Traffic lights	0 → no traffic light	
⋮	⋮	

$$Y = \begin{bmatrix} | & | & | & & | \\ y^{(1)} & y^{(2)} & y^{(3)} & \cdots & y^{(m)} \\ | & | & | & & | \end{bmatrix}_{(4,m)}$$

Andrew Ng

Neural network architecture



Loss : $\hat{y}^{(i)}$ $(4,1)$

$$\frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{4} \mathcal{L}(\hat{y}_j^{(i)}, y_j^{(i)})$$

↳ usual logistic loss

$$-y_j^{(i)} \log \hat{y}_j^{(i)} - (1 - y_j^{(i)}) \log (1 - \hat{y}_j^{(i)})$$

Unlike softmax regression:

- one image can have multiple labels

Multi-task learning

Andrew Ng

- Difference between multitask learning and softmax regression:
  - Softmax regression assigns a single label to a single example.
  - In multitask learning, one image can have multiple labels. We go through the different classes and ask for each of the classes, does that type of object appear in the image?

- If you train a neural network to minimize this cost function, you are carring out multi-task learning.
  - NN looks at each image and solves four problems:
  - It is trying to tell if each image has each of the 4 objects in it.
- You could have trained 4 different NNs to recognize 4 different objects:
  - Some of the earlier features can be shared between there different typer of objects
  - Training 1 NN to do 4 things gives better performance than training 4 completely seperate neural networks.

- Multi-task learning also works even if some of the images will have labels for some of the objects:

$$Y = \begin{bmatrix} 1 & 1 & 0 & ? \\ 0 & 1 & \cdots 1 \cdots & 1 \\ ? & ? & 1 & ? \\ ? & ? & 0 & ? \end{bmatrix}$$

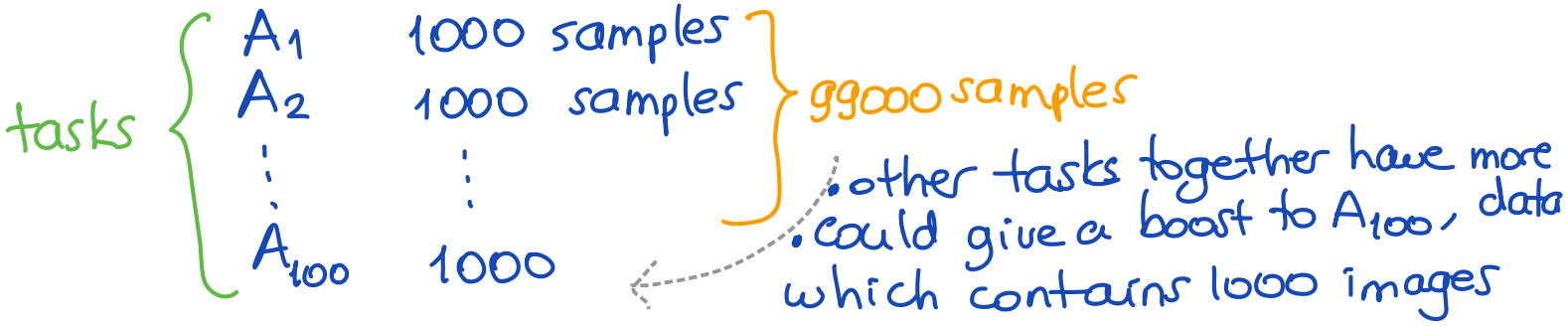
When training with missing labels:

$$\frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{4} \mathcal{L}(\hat{y}_j^{(i)}, y_j^{(i)})$$

↳ sum only over values of j with 0/1 label

When multi-task learning makes sense

- Training on a set of tasks that could benefit from having shared lower-level features.
- Usually: Amount of data you have for each task is quite similar.



- Can train a big enough neural network to do well on all the tasks.

Andrew Ng

Multi-task learning

Multi-task learning refers to having one neural network do simultaneously several tasks.

When to use multi-task learning

- Training on a set of tasks that could benefit from having shared lower-level features
- Usually: Amount of data you have for each task is quite similar
- Can train a big enough neural network to do well on all tasks

Example: Simplified autonomous vehicle

The vehicle has to detect simultaneously several things: pedestrians, cars, road signs, traffic lights, cyclists, etc. We could have trained four separate neural networks, instead of train one to do four tasks. However, in this case, the performance of the system is better when one neural network is trained to do four tasks than training four separate neural networks since some of the earlier features in the neural network could be shared between the different types of objects.

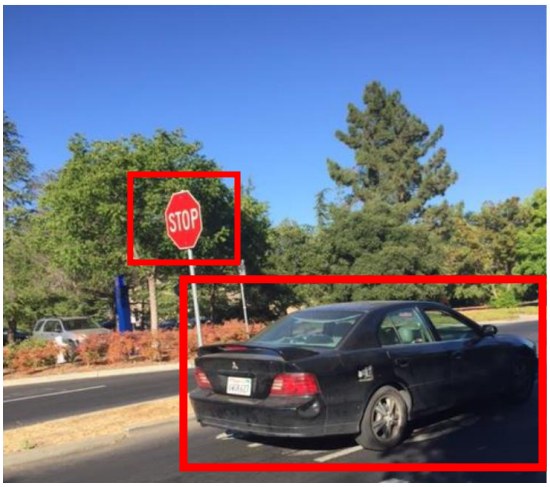
The input $x^{(i)}$ is the image with multiple labels
The output $y^{(i)}$ has 4 labels which are represents:

$$y^{(i)} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \begin{matrix} \text{Pedestrians} \\ \text{Cars} \\ \text{Road signs - Stop} \\ \text{Traffic lights} \end{matrix}$$

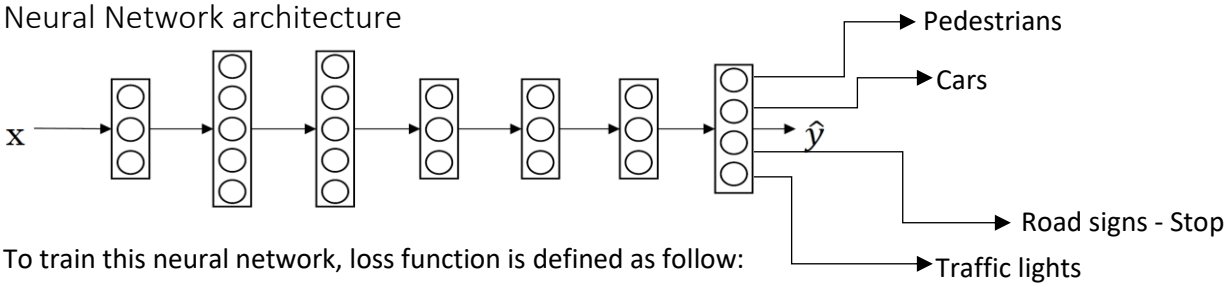
$$Y = \begin{bmatrix} | & | & | & | \\ y^{(1)} & y^{(2)} & y^{(3)} & y^{(4)} \\ | & | & | & | \end{bmatrix}$$

$$Y = (4, m)$$

$$Y = (4,1)$$



Neural Network architecture



To train this neural network, loss function is defined as follow:

$$-\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{4}\left(y_j^{(i)}\log\left(\hat{y}_j^{(i)}\right)+\left(1-y_j^{(i)}\right)\log\left(1-\hat{y}_j^{(i)}\right)\right)$$

Also, the cost can be compute such as it is not influenced by the fact that some entries are not labeled.
Example:

$$Y = \begin{bmatrix} 1 & 0 & ? & ? \\ 0 & 1 & ? & 0 \\ 0 & 1 & ? & 1 \\ ? & 0 & 1 & 0 \end{bmatrix}$$



deeplearning.ai

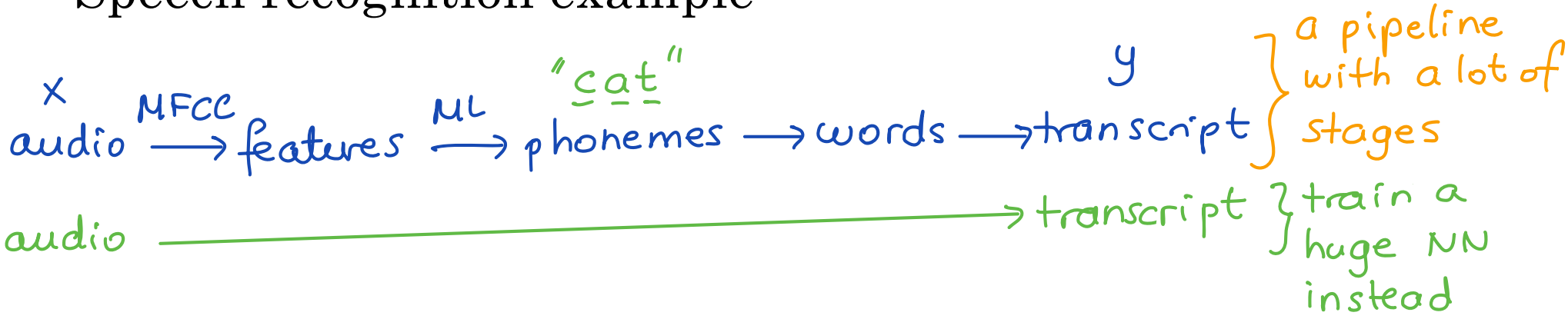
End-to-end deep learning

What is end-to-end deep learning

- Some data processing systems or learning systems require multiple stages of processing.
- End-to-end learning takes all those multiple stages and replaces with just a single neural network.

What is end-to-end learning?

Speech recognition example



- You can train a single NN to learn the function mapping from x to y directly bypassing the intermediate steps, which may give better results.
- End-to-end learning has obsoleted many years of research in some of the intermediate components.
- One of the challenges of deep learning is that you might need a lot of data before it works well.

Andrew Ng

- If you have 3000 hours of data → traditional method works really well.
- If you have 10.000 hours of data (up to 100.000 hours) then end-to-end methods work really well
- If you have medium amount of data, there are also intermediate approaches:

audio ——→ phonemes ——→ transcript

This is a step towards end-to-end learning

Face recognition



[Image courtesy of Baidu]

- Rather than needing to swipe an RFID badge to enter the facility, approach the camera and if the person is recognized, it lets you through.
- How do you build a system like this?

Image captured by camera

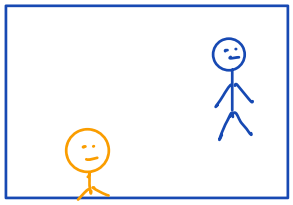
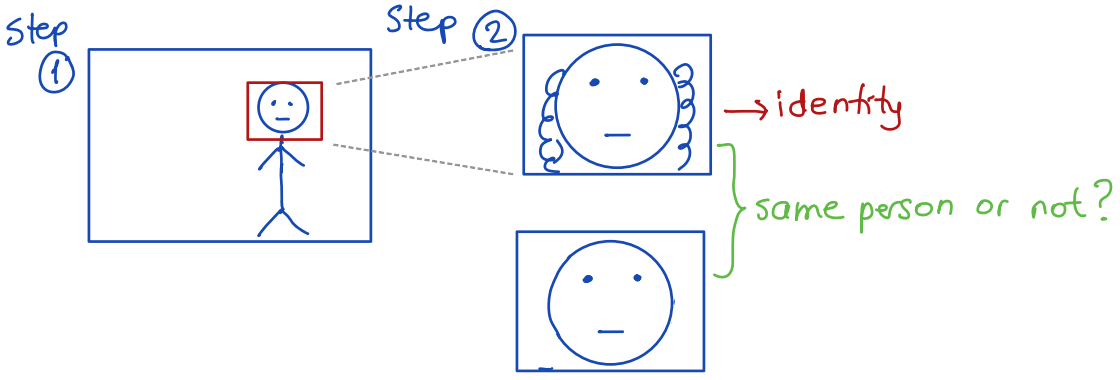


Image (x) ⟶ y (identity)

- As the person approaches the turnstile the person will appear bigger.

- A multi-step approach is better.
  - First, detect the person's face and zoom in to that part
  - Crop the image so that person's face is centered.
  - Feed it to a NN to learn or estimate the person's identity.



- Why does the 2-step approach work better?
  - Reason 1: each of the two problems you're solving is actually much simpler.
  - Reason 2: you have a lot of data for each of the two subtasks. (face detection and face recognition)
- We don't have enough data for the end-to-end approach.

More examples

Machine translation

Traditional approach: English → text analysis → ---- → French

End-to-end: English → French

(x, y)

English

French

Estimating child's age:



X-ray

Multi-stage approach: Image → (1) bones → (2) age

End-to-end approach: Image → age

doesn't work well if there's not enough training data

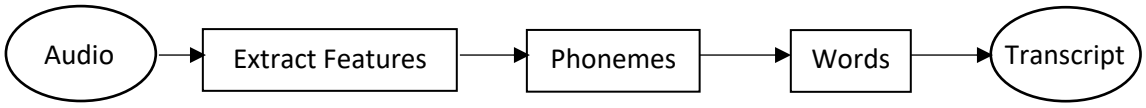
Andrew Ng

What is end-to-end deep learning

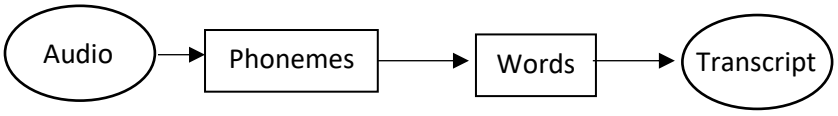
End-to-end deep learning is the simplification of a processing or learning systems into one neural network.

Example - Speech recognition model

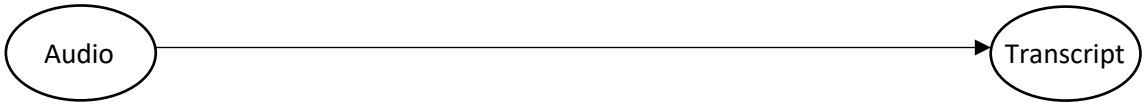
The traditional way - small data set



The hybrid way - medium data set



The End-to-End deep learning way – large data set



End-to-end deep learning cannot be used for every problem since it needs a lot of labeled data. It is used mainly in audio transcripts, image captures, image synthesis, machine translation, steering in self-driving cars, etc.



deeplearning.ai

End-to-end deep learning

Whether to use end-to-end learning

- Benefits of end-to-end learning
  1. Lets the data speak: If you have enough (X, y) data, whatever is the most appropriate function mapping from X to y, if you train a big enough neural network, hopefully the network will figure it out.
     - Network will capture the statistics in the data, rather than being forced to reflect human preconceptions.

     e.g. Earlier speech recognition systems had the notion of a phoneme which was a basic unit of sound (c a t : 3 phonemes). Phonemes are developed by linguists and are a reasonable description of language, but it's not obvious that we want to force the ML algorithm to recognize phonemes. If we let the algorithm to learn whatever representation it wants to learn, its performance may be better.
  2. There's less hand designing of components needed.

     - simplifies the design process.

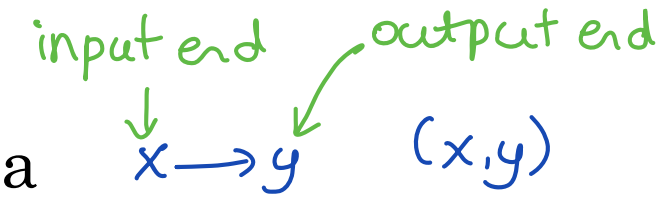
Pros and cons of end-to-end deep learning

Pros:

- Let the data speak
- Less hand-designing of components needed

Cons:

- May need large amount of data



- Excludes potentially useful hand-designed components: if you don't have enough data, then you can inject manual knowledge into the algorithm.
  - A learning algorith has two main sources of knowledge: (i) data and (ii) hand-designed features.

Andrew Ng

Applying end-to-end deep learning

Key question: Do you have sufficient data to learn a function of the complexity needed to map x to y?

$x \rightarrow y$



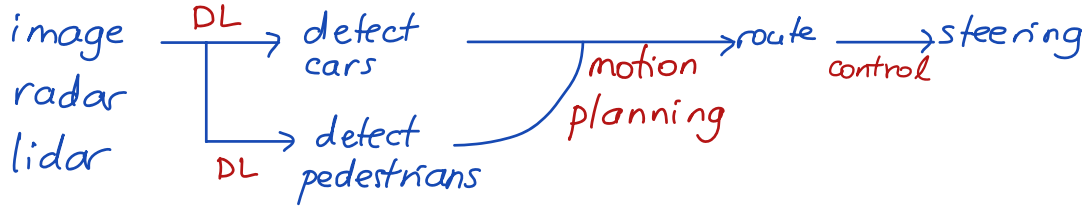
→ age

- Mapping the hand x-ray to the age of the child is a more complex task as compared to detecting the bones in the image.

Example: Autonomous driving

How do you build a car that drives itself?

e.g. (not an end-to-end approach)



- Use DL to learn individual components
- Carefully choose $X \rightarrow Y$ depending what tasks you can get data for.

- Using end-to-end learning is not the most promising approach given the data availability

image ⟶ steering

Whether to use end-to-end deep learning

Before applying end-to-end deep learning, you need to ask yourself the following question: Do you have enough data to learn a function of the complexity needed to map x and y?

Pro:

- Let the data speak
  - By having a pure machine learning approach, the neural network will learn from x to y. It will be able to find which statistics are in the data, rather than being forced to reflect human preconceptions.

- Less hand-designing of components needed
  - It simplifies the design work flow.

Cons:

- Large amount of labeled data
  - It cannot be used for every problem as it needs a lot of labeled data.

- Excludes potentially useful hand-designed component
  - Data and any hand-design's components or features are the 2 main sources of knowledge for a learning algorithm. If the data set is small than a hand-design system is a way to give manual knowledge into the algorithm.